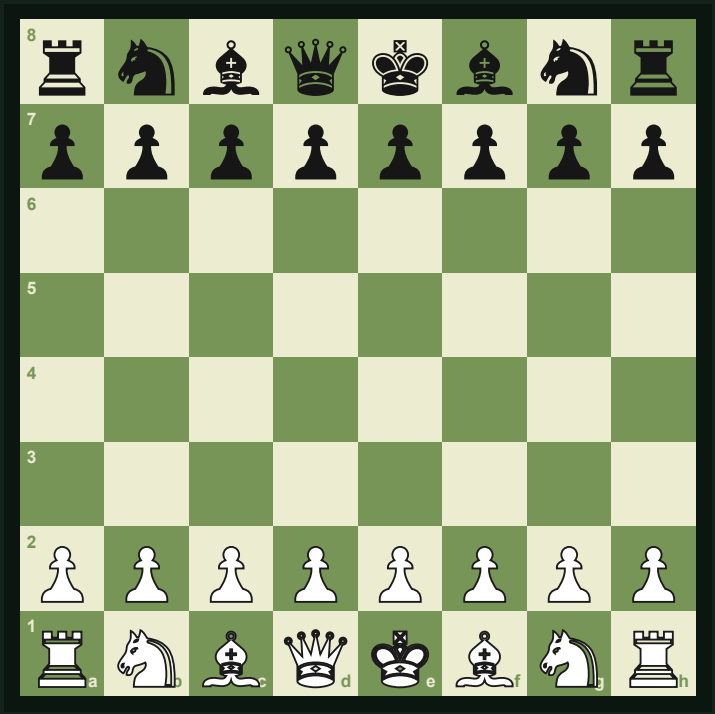


A LEARNING ROADMAP

Building a Chess Engine in C++

From human vs human to a real AI, one phase at a time

Pseudocode only. You write the real code.

November 2026

START HERE

How to use this guide

This guide takes you from an empty `main.cpp` to a chess program you can play against, in nine phases. Every phase ends with a program that runs, so you always have something working.

- **Go in order.** Each phase uses the one before it.
- **Pseudocode isn't C++.** It shows the idea. Turning it into real code is where the learning happens. Each phase lists the C++ features you'll need under *C++ to look up*.
- **Don't move on until "Done when" is true.** Bugs from early phases get much harder to find later.
- **Commit every time something small works.** Appendix B shows how.

> **THE ONE BIG IDEA**
> Build everything around one function: *give me every legal move in this position*. For human vs human, a typed move is allowed only if it's in that list. For the AI, the computer picks the best move from the same list. Built this way, adding the AI later means adding a new file, not rewriting what you have.

Contents

Phase 0: Setup and your first commit . . . 3

Phase 1: The board . . . 4

Phase 2: Human vs human loop . . . 5

Phase 3: How each piece moves . . . 6

Phase 4: Check, checkmate and stalemate . . . 8

Phase 5: Special rules . . . 10

Phase 6: Prove it's correct with perft . . . 12

Phase 7: The AI, in small steps . . . 14

Phase 8: Optimizations . . . 16

Appendix A: Growing into multiple files . . . 17

Appendix B: Git and GitHub . . . 19

Appendix C: Glossary . . . 21

PHASE 0

Setup and your first commit

GOAL	Build and run a program from VS Code, and get your code onto GitHub.
DONE WHEN	**Cmd+Shift+B** builds with no errors, `./main` prints Hello World, and you can see `main.cpp` on github.com/iwe556/chess-engine.

You've already done most of this. VS Code is set up so **Cmd+Shift+B** builds `main.cpp` with `clang++` into a program called `main`. To run it, open a terminal in VS Code (**Terminal → New Terminal**) and type:

```
./main
```

Your project also has a `.gitignore` that keeps the built program out of git, and quieter IntelliSense settings in `.vscode/settings.json`.

The only thing left is your first commit and push. Follow **Appendix B, steps 1 to 4**.

C++ TO LOOK UP `#include <iostream>` · `int main()` · `std::cout` · `"\n"`

COMMIT POINT `git commit -m "Hello world"`

PHASE 1

The board

GOAL	Store a chess position in memory and print it.
DONE WHEN	The program prints the starting position with rank numbers down the side and file letters along the bottom.

The easiest way to start is an 8×8 grid of characters:

- Uppercase letters are White: `P N B R Q K` (pawn, knight, bishop, rook, queen, king).
- Lowercase letters are Black: `p n b r q k`.
- A dot `.` is an empty square.
- `board[row][col]`: row 0 is **rank 8** (the top of the printed board), and col 0 is the **a-file**.

```
struct Position:
    board[8][8] of char
    sideToMove                  // 'w' or 'b'

function setupStartPosition(pos):
    row 0     = "rnbqkbnr"      // Black's back rank (rank 8)
    row 1     = all 'p'
    rows 2..5 = all '.'
    row 6     = all 'P'
    row 7     = "RNBQKBNR"      // White's back rank (rank 1)
    pos.sideToMove = 'w'

function printBoard(pos):
    for row from 0 to 7:
        print (8 - row) and a space          // the rank number
        for col from 0 to 7:
            print pos.board[row][col] and a space
        print a newline
    print "  a b c d e f g h"
```

Square names ↔ grid positions

You'll use this conversion constantly, so it's worth understanding why it works. Characters are stored as numbers, and the letters `a` to `h` (and digits `1` to `8`) are next to each other, so subtracting gives a position.

```
col = letter - 'a'           // 'e' - 'a' = 4
row = 8 - (digit - '0')      // '2' - '0' = 2, and 8 - 2 = 6

// and back again, for printing a move:
letter = 'a' + col
digit  = '0' + (8 - row)
```

WATCH OUT

Rank 1 is **row 7**, not row 0. Mixing these up is the most common bug in this phase. Print the board after every change and check e2 really is a white pawn.

C++ TO LOOK UP `struct` · `char board[8][8]` · `char arithmetic` · `std::cout` · `for loops`

COMMIT POINT `git commit -m "Print the starting position"`

PHASE 2

Human vs human loop

GOAL	Two people can take turns typing moves.
DONE WHEN	You can type `e2e4`, see the pawn move, and the program says it's Black's turn. Pieces can move anywhere for now.

A move is just two squares, plus an optional promotion piece for later:

```
struct Move:
    fromRow, fromCol
    toRow, toCol
    promotion                  // '.' if none, otherwise 'q', 'r', 'b' or 'n'

function parseMove(text) -> Move, or "invalid":
    if text length is not 4 (or 5 with a promotion letter): invalid
    if text[0], text[2] aren't 'a'..'h' or text[1], text[3] aren't '1'..'8': invalid
    fromCol = text[0] - 'a';   fromRow = 8 - (text[1] - '0')
    toCol   = text[2] - 'a';   toRow   = 8 - (text[3] - '0')
    promotion = text[4] if there is one, else '.'
```

Moving a piece is two assignments. Notice that `makeMove` also hands the turn to the other side. Keeping that inside `makeMove` matters later: perft and the AI both rely on it.

```
function makeMove(pos, move):
    pos.board[toRow][toCol]     = pos.board[fromRow][fromCol]
    pos.board[fromRow][fromCol] = '.'
    pos.sideToMove = the other side
```

The game loop

```
setupStartPosition(pos)
loop forever:
    printBoard(pos)
    print whose turn it is
    line = read a line of input                // e.g. "e2e4"
    if line is "quit": stop
    move = parseMove(line)
    if move is invalid: print "Bad input, try e2e4", start the loop again
    piece = pos.board[move.fromRow][move.fromCol]
    if piece is '.' or the wrong color: print "Not your piece", start again
    makeMove(pos, move)
```

At this point pieces teleport anywhere. That's expected: movement rules come in Phase 3.

> **WATCH OUT**
> Check the input **before** converting it. If someone types `z9z9` and you convert it anyway, you'll read outside the array, which can crash or silently corrupt memory in C++.

C++ TO LOOK UP `std::string` · `std::getline(std::cin, line)` · `line.size()` · `isupper / islower` · `continue`

COMMIT POINT `git commit -m "Players can take turns"`

PHASE 3

How each piece moves

GOAL	Generate every move each piece can make, following its movement rules.
DONE WHEN	Impossible moves are rejected: a knight moving like a bishop, a rook jumping over a piece, a pawn moving backwards.

Don't write `isMoveValid()`. Write `generateMoves()`, which returns a list of **every** move for the side to move. At this stage moves follow each piece's movement rules but ignore check; that comes in Phase 4. Engine programmers call these *pseudo-legal* moves.

The trick: pieces are lists of directions

```
KNIGHT_JUMPS = (−2,−1) (−2,1) (−1,−2) (−1,2) (1,−2) (1,2) (2,−1) (2,1)
KING_STEPS   = all 8 neighboring squares
ROOK_DIRS    = (−1,0) (1,0) (0,−1) (0,1)
BISHOP_DIRS  = (−1,−1) (−1,1) (1,−1) (1,1)
QUEEN_DIRS   = ROOK_DIRS + BISHOP_DIRS
```

Knights and kings **jump** once in each direction. Rooks, bishops and queens **slide**, repeating a direction until something blocks them. So you only need two helpers for five piece types, plus a special one for pawns.

```
function generateMoves(pos) -> list of Move:
    moves = empty list
    for each square (r, c) holding a piece of pos.sideToMove:
        if knight: addJumps(pos, r, c, KNIGHT_JUMPS, moves)
        if king:   addJumps(pos, r, c, KING_STEPS, moves)
        if rook:   addSlides(pos, r, c, ROOK_DIRS, moves)
        if bishop: addSlides(pos, r, c, BISHOP_DIRS, moves)
        if queen:  addSlides(pos, r, c, QUEEN_DIRS, moves)
        if pawn:   addPawnMoves(pos, r, c, moves)
    return moves

function addJumps(pos, r, c, offsets, moves):
    for each (dr, dc) in offsets:
        target = (r + dr, c + dc)
        if target is off the board: skip it
        if target is empty or holds an enemy piece: add the move

function addSlides(pos, r, c, directions, moves):
    for each (dr, dc) in directions:
        step = 1
        loop:
            target = (r + dr*step, c + dc*step)
            if target is off the board: stop this direction
            if target is empty:      add the move, step = step + 1
            else if enemy piece:     add the move (a capture), stop this direction
            else (your own piece):   stop this direction
```

Pawns are the odd ones out

```
function addPawnMoves(pos, r, c, moves):
    forward  = −1 if white, +1 if black       // white moves UP the array
    startRow =  6 if white,  1 if black
    lastRow  =  0 if white,  7 if black

    if (r + forward, c) is empty:
        add the move
        if r == startRow and (r + 2*forward, c) is also empty:
            add the double push
    for dc in (−1, +1):
        if (r + forward, c + dc) holds an enemy piece: add the capture

    // any pawn move landing on lastRow becomes FOUR moves:
    // promote to queen, rook, bishop or knight
```

Now change the game loop: if the typed move isn't in `generateMoves(pos)`, reject it. You'll need a way to compare two moves, such as a `sameMove(a, b)` function that checks all five fields.

> **WATCH OUT**
> Check the target is **on the board before** reading the square. With sliding pieces, it's easy to step to column 8 or row -1 and read memory that isn't part of the board.

C++ TO LOOK UP `std::vector<Move>` · `push_back` · `pass by reference (&)` · `const` · `functions returning bool`

COMMIT POINT `git commit -m "Add knight moves"` *(one commit per piece type)*

PHASE 4

Check, checkmate and stalemate

GOAL	No move may leave your own king in check, and the game knows when it's over.
DONE WHEN	Pinned pieces can't move, you must get out of check, and Fool's Mate (`f2f3`, `e7e5`, `g2g4`, `d8h4`) ends the game with "Checkmate".

One helper does most of the work. The trick is to stand on the square and look **outward**, pretending to be each piece type. If a knight on this square could reach an enemy knight, that enemy knight can reach this square too.

```
function isSquareAttacked(pos, r, c, byColor) -> true or false:
    for each (dr, dc) in KNIGHT_JUMPS:
        if (r+dr, c+dc) is on the board and holds byColor's knight: return true
    for each direction in ROOK_DIRS:
        slide outward until you hit a piece
        if it is byColor's rook or queen: return true
    for each direction in BISHOP_DIRS:
        slide outward until you hit a piece
        if it is byColor's bishop or queen: return true
    for each (dr, dc) in KING_STEPS:
        if (r+dr, c+dc) holds byColor's king: return true
    pawnRow = r + 1 if byColor is white, r - 1 if black
    if (pawnRow, c-1) or (pawnRow, c+1) holds byColor's pawn: return true
    return false

function inCheck(pos, color) -> true or false:
    find the square (kr, kc) of color's king
    return isSquareAttacked(pos, kr, kc, the other color)
```

From pseudo-legal to legal

Try each move on a copy of the board. If it leaves your own king attacked, throw it away.

```
function generateLegalMoves(pos) -> list of Move:
    legal = empty list
    mover = pos.sideToMove
    for each move in generateMoves(pos):
        copy = pos                          // copy the whole position
        makeMove(copy, move)
        if not inCheck(copy, mover): add move to legal
    return legal

// at the top of each turn in the game loop:
moves = generateLegalMoves(pos)
if moves is empty:
    if inCheck(pos, pos.sideToMove): print "Checkmate!", stop
    else:                            print "Stalemate, it's a draw", stop
if inCheck(pos, pos.sideToMove): print "Check!"
```

The game loop should now accept a typed move only if it's in `generateLegalMoves(pos)`. Copying the whole board for every move is slow, but it's simple and correct. Replacing it is one of the optimizations in Phase 8.

> **WATCH OUT**
> Pawns attack in the direction they **move**. A white pawn attacks upward, so to find a white pawn attacking (r, c), look one row **below** it: row r + 1.

C++ TO LOOK UP `Position copy = pos;` · `bool` · `returning early from loops`

COMMIT POINT `git commit -m "Legal moves, check and checkmate"`

PHASE 5

Special rules

GOAL	Castling, en passant and promotion all work.
DONE WHEN	You can play each one in a game, and the perft numbers in Phase 6 match.

These rules need the position to remember a little more than where the pieces are:

```
struct Position:
    board[8][8], sideToMove                 // from Phase 1
    whiteKingside, whiteQueenside           // castling rights: true/false each
    blackKingside, blackQueenside
    enPassantSquare                         // square a pawn just skipped over, or none
    halfmoveClock                           // for the 50-move draw rule
```

Castling

Add this to the king's moves in `generateMoves`. It uses `isSquareAttacked` from Phase 4.

```
// White kingside
if whiteKingside and f1, g1 are empty
   and e1, f1, g1 are NOT attacked by black:
       add the move e1 -> g1

// White queenside
if whiteQueenside and b1, c1, d1 are empty
   and e1, d1, c1 are NOT attacked by black:
       add the move e1 -> c1

// Black: the same idea on rank 8 (e8, f8, g8 and so on)
```

En passant

When a pawn moves two squares, remember the square it skipped. On the very next move only, an enemy pawn may capture onto that square. In `addPawnMoves`, alongside normal captures:

```
if (r + forward, c + dc) is pos.enPassantSquare: add the capture
```

Updating makeMove

`makeMove` now has more bookkeeping. Order matters: the en passant check must happen **before** the piece moves, while the target square is still empty.

```
function makeMove(pos, move):
    piece = the piece on the from-square

    // castling: a king moving two columns also moves its rook
    if piece is a king and it moves 2 columns:
        move the rook from its corner to the square the king passes over

    // en passant: a pawn moving diagonally onto an EMPTY square
    if piece is a pawn and fromCol != toCol and the target is empty:
        remove the enemy pawn at (fromRow, toCol)

    move the piece as before
    if move.promotion is not '.':
        put the promotion piece (in the mover's case) on the target square

    // bookkeeping for the next move
    enPassantSquare = none
    if piece is a pawn and it moved 2 rows:
        enPassantSquare = the square it skipped over
    if piece is a king: lose both of that side's castling rights
    if anything moves FROM or TO a1, h1, a8 or h8: lose that corner's right
    halfmoveClock = 0 if a pawn moved or a piece was captured, else + 1
    sideToMove = the other side
```

The "from or to a corner" rule is a neat shortcut: it covers both a rook moving away and a rook being captured, without tracking rooks separately.

> **WATCH OUT**
> In queenside castling, **b1 must be empty but may be attacked.** Only the squares the king stands on or crosses (e1, d1, c1) have to be safe.

Draw rules (optional for now)

- **50-move rule:** if `halfmoveClock` reaches 100 (50 moves by each side), it's a draw.
- **Not enough material:** king vs king, or king and one bishop or knight vs king, is a draw.
- **Threefold repetition:** save a text snapshot of each position. If one appears three times, it's a draw. Phase 8's Zobrist hashing makes this much faster.

C++ TO LOOK UP `more struct fields` · `std::abs` · `enum (optional)`

COMMIT POINT `git commit -m "Add castling"` *(then en passant, then promotion)*

PHASE 6

Prove it's correct with perft

GOAL	Be certain your move generation has no bugs.
DONE WHEN	All three test positions below match the published numbers.

Perft (short for *performance test*) counts every position reachable in exactly N moves. Engine writers have computed these numbers precisely, so if yours match, your rules are right. If they don't, you have a bug, even if every game you've played looked fine.

```
function perft(pos, depth) -> number:
    if depth == 0: return 1
    total = 0
    for each move in generateLegalMoves(pos):
        copy = pos
        makeMove(copy, move)
        total = total + perft(copy, depth - 1)
    return total
```

Loading test positions with FEN

The starting position barely tests castling, en passant or promotion, so you need to load other positions. **FEN** is the standard one-line text format for a chess position. Here's the starting position:

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

- **Pieces**, rank 8 first, ranks separated by `/`. A digit means that many empty squares.
- **Side to move:** `w` or `b`.
- **Castling rights:** any of `KQkq`, or `-` for none.
- **En passant square**, like `e3`, or `-` for none.
- **Halfmove clock** and **move number**.

```
function loadFEN(pos, text):
    fields = split text on spaces
    row = 0, col = 0
    for each character ch in fields[0]:
        if ch is '/':      row = row + 1, col = 0
        else if ch is a digit:
            fill that many squares with '.', col = col + that digit
        else:              pos.board[row][col] = ch, col = col + 1
    pos.sideToMove = fields[1]
    castling rights = which of K, Q, k, q appear in fields[2]
    pos.enPassantSquare = fields[3]          // '-' means none
```

The numbers to match

Position	Depth 1	Depth 2	Depth 3	Depth 4	Depth 5
Start position	20	400	8,902	197,281	4,865,609
Kiwipete	48	2,039	97,862	4,085,603	
Position 3	14	191	2,812	43,238	674,624

```
// Kiwipete: castling, pins and promotions everywhere
r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBBPPP/R3K2R w KQkq - 0 1

// Position 3: tricky en passant and pins in an endgame
8/2p5/3p4/KP5r/1R3p1k/8/4P1P1/8 w - - 0 1
```

When a number is wrong: divide

Print the perft count under each first move separately. Then compare with a program you know is correct: the free engine Stockfish prints this exact breakdown if you type `position fen <your FEN>` and then `go perft 3`. Whichever move has a different count is where the bug is. Make that move and divide again, one level deeper.

```
function divide(pos, depth):
    for each move in generateLegalMoves(pos):
        copy = pos
        makeMove(copy, move)
        print the move as text, ": ", perft(copy, depth - 1)
```

Depth 5 can take a while while you're still copying the whole board. That's normal; building with `-O2` (Appendix A) helps a lot.

> **MILESTONE**
> When all three positions match, your rules are correct and **human vs human is done.** Push it to GitHub. Everything after this is the AI.

C++ TO LOOK UP `recursion` · `long long` · `std::stringstream` · `isdigit`

COMMIT POINT `git commit -m "Perft matches for all test positions"`

PHASE 7

The AI, in small steps

GOAL	Play against the computer.
DONE WHEN	At depth 3 the AI stops giving away pieces for free, and alpha-beta returns the same scores as minimax, only faster.

Step 1: A random mover

This proves the plumbing works: the AI can pick a move, and the game loop can play it.

```
function randomMove(pos) -> Move:
    moves = generateLegalMoves(pos)
    return moves[a random number from 0 to moves.size - 1]
```

Step 2: Evaluation

A score for how good a position is. **Positive means good for White, negative means good for Black.** Values are in *centipawns*: 100 means one pawn.

```
PIECE_VALUE: pawn 100, knight 300, bishop 300, rook 500, queen 900, king 0

function evaluate(pos) -> number:
    score = 0
    for each square holding a piece:
        if the piece is white: score = score + PIECE_VALUE[piece]
        else:                  score = score - PIECE_VALUE[piece]
    return score
```

Step 3: Greedy, looking one move ahead

```
function greedyMove(pos) -> Move:
    for each move in generateLegalMoves(pos):
        copy = pos
        makeMove(copy, move)
        score = evaluate(copy)
        keep the move if its score is the best so far for the side moving
        // White wants the HIGHEST score, Black wants the LOWEST
    return the move you kept
```

It will grab pieces, but it will also take a defended pawn with its queen and lose the queen next move. It can't see your reply. That's what minimax fixes.

Step 4: Minimax

Look several moves ahead: "I pick my best move, assuming you'll pick your best reply, assuming I'll pick my best reply to that..."

```
MATE = 100000

function minimax(pos, depth) -> number:
    moves = generateLegalMoves(pos)
    if moves is empty:
        if inCheck(pos, pos.sideToMove):            // checkmate
            return -MATE if White is to move, else +MATE
        return 0                                    // stalemate
    if depth == 0: return evaluate(pos)

    if White is to move:                            // White maximizes
        best = -infinity
        for each move: best = max(best, minimax(child, depth - 1))
    else:                                           // Black minimizes
        best = +infinity
        for each move: best = min(best, minimax(child, depth - 1))
    return best
    // "child" means: a copy of pos with that move made, like in perft

function findBestMove(pos, depth) -> Move:
    for each move in generateLegalMoves(pos):
        score = minimax(child, depth - 1)
        remember the move if score is the best so far for pos.sideToMove
    return the remembered move
```

Step 5: Alpha-beta

Alpha-beta gives exactly the same answer as minimax, but skips lines that can't change the result. **alpha** is the best score White is already guaranteed somewhere else; **beta** is the best Black is guaranteed. If a line is already worse for one side than something they can force elsewhere, they'll never allow it, so stop exploring it.

```
function alphabeta(pos, depth, alpha, beta) -> number:
    same "no moves" and "depth == 0" checks as minimax

    if White is to move:
        for each move:
            score = alphabeta(child, depth - 1, alpha, beta)
            alpha = max(alpha, score)
            if alpha >= beta: break        // Black would never allow this line
        return alpha
    else:
        for each move:
            score = alphabeta(child, depth - 1, alpha, beta)
            beta = min(beta, score)
            if alpha >= beta: break        // White would never allow this line
        return beta

// in findBestMove, call: alphabeta(child, depth - 1, -infinity, +infinity)
```

A great test: for the same position and depth, minimax and alpha-beta must return the same score. If they don't, alpha-beta has a bug.

Step 6: Add a menu

```
print "1) Human vs Human   2) Human vs AI"
read the choice
then, on every turn:
    if it's the AI's turn: move = findBestMove(pos, 3)
    else:                  read the move from the keyboard, as before
```

WATCH OUT

If your AI seems to choose the **worst** move on purpose, you've almost certainly flipped a sign: maximizing where you should minimize. Test with a position where Black can take a free queen and check that Black takes it.

C++ TO LOOK UP `recursion` · `std::max / std::min` · `<random>` · `std::numeric_limits<int>::max()`

COMMIT POINT `git commit -m "Add alpha-beta AI"`

PHASE 8

Optimizations

Only start these once Phase 7 works. Each row is its own small project: pick one, build it, measure it, commit it. They're roughly in the order most people tackle them.

The Chess Programming Wiki (chessprogramming.org) has a page for every row in this table. It's the best reference there is for engine programming.

Optimization	Effort	What it gets you
Negamax	Easy	Merges the White and Black halves of your search into one function by flipping the sign of the score.
Move ordering	Easy	Try captures first (best victim, cheapest attacker). Alpha-beta then skips far more lines, so the same depth runs much faster.
Piece-square tables	Easy	A bonus or penalty for each piece on each square: knights like the center, pawns like advancing. Play becomes much more natural.
Mate distance	Easy	Return MATE minus the number of moves so far, so the AI picks the fastest checkmate instead of a slow one.
Iterative deepening + time limit	Medium	Search depth 1, then 2, then 3, until time runs out. There's always a move ready.
Quiescence search	Medium	At depth 0, keep searching captures only. Stops the AI misjudging a position in the middle of a trade (the "horizon effect").

Optimization	Effort	What it gets you
Make / unmake	**Medium**	Undo a move instead of copying the whole board. A big speed boost, and perft will catch any mistakes.
0x88 or 10×12 board	**Medium**	Board layouts that make "is this off the board?" checks much cheaper.
UCI protocol	**Medium**	The standard way engines talk to chess apps like Cute Chess or Arena, so you can play matches against other engines.
Zobrist hashing + transposition table	**Hard**	Remembers positions it has already searched. Also gives fast threefold repetition detection.
Bitboards	**Hard**	One 64-bit number per piece type. This is how strong engines store the board.

> **THE GOLDEN RULE**
> After every optimization: **(1)** rerun perft if you touched move generation or `makeMove`, and **(2)** have the new version play the old one over many games to prove it's actually stronger. A faster engine that plays worse is a bug.

APPENDIX A

Growing into multiple files

Start with everything in `main.cpp`. Once it passes about 300 lines, split it up so each file has one job:

File	What goes in it
`main.cpp`	The menu and the game loop
`board.h` / `board.cpp`	`Position`, `setupStartPosition`, `printBoard`, `makeMove`, `loadFEN`
`movegen.h` / `movegen.cpp`	`generateMoves`, `generateLegalMoves`, `isSquareAttacked`, `perft`
`search.h` / `search.cpp`	`evaluate`, `minimax` / `alphabeta`, `findBestMove` (Phase 7)

Headers vs source files

A `.h` file says **what exists**: struct definitions and function signatures. A `.cpp` file says **how it works**. Other files `#include` the header so they know a function exists, without needing its code.

```
// board.h: WHAT exists
#pragma once                              // stops the file being included twice
struct Position { ... };
void printBoard(const Position& pos);     // a declaration: ends with ;

// board.cpp: HOW it works
#include "board.h"
void printBoard(const Position& pos) { ...the real code... }
```

Building more than one file

VS Code's build task currently builds only the file you have open. In `.vscode/tasks.json`, in the `clang++` task, change `"${file}"` to `"${fileDirname}/*.cpp"`. Or build from the terminal:

```
clang++ -std=c++17 -Wall -O2 -o main *.cpp
```

`-O2` tells the compiler to optimize, which makes perft and the AI several times faster. Leave it out while you're stepping through code in the debugger, because it rearranges your code and makes debugging confusing.

C++ TO LOOK UP `header files` · `#pragma once` · `#include "file.h"` · `declarations vs definitions`

APPENDIX B

Git and GitHub

Your folder is already a git repository linked to github.com/iwe556/chess-engine, and it already has a `.gitignore`. So skip `git init` and `git remote add`; start here.

1. Tell git who you are (once per computer)

```
git config --global user.name "Your Name"
git config --global user.email "the-email-on-your-github-account"
```

2. Make your first commit

```
git add .
git commit -m "Hello world"
```

`git add .` stages every changed file. `git commit` saves a snapshot with a message describing it.

3. Log in to GitHub

GitHub won't accept your normal account password when you push. The easiest fix is the GitHub command-line tool, which logs you in through your web browser:

```
brew install gh
gh auth login
```

When it asks, choose **GitHub.com**, then **HTTPS**, then **Login with a web browser**. If you don't have Homebrew, create a *Personal Access Token* on GitHub instead (Settings → Developer settings) and paste it when git asks for a password.

4. Push

```
git push -u origin main
```

`-u` links your local `main` branch to GitHub's. After this first time, `git push` is enough.

Your everyday routine

```
git status                          # what changed since your last commit?
git add .
git commit -m "Add knight moves"
git push
```

Write commit messages that say what changed, like *Add knight moves* or *Fix pawn direction for Black*, not *stuff* or *fix*. Future you will thank you.

Branches, for experiments

When you try something risky, like a new search feature, do it on a branch so `main` always works. If the experiment works out, merge it in. If not, just switch back.

```
git switch -c alphabeta             # make a new branch and move to it
...work and commit as usual...
git switch main                     # go back to main
git merge alphabeta                 # bring the branch's commits into main
git push
```

Handy commands

Command	What it does
`git status`	Shows which files changed since your last commit
`git diff`	Shows the exact lines you changed
`git log --oneline`	Lists your commits, newest first
`git restore main.cpp`	Throws away uncommitted changes to a file. **This can't be undone.**
`git switch -c name`	Creates a new branch and moves to it
`git switch main`	Moves back to the main branch
`git merge name`	Brings a branch's commits into the branch you're on

APPENDIX C

Glossary

Ply	One move by one side. "Depth 4" means 4 plies: two moves each.
Pseudo-legal move	A move that follows how the piece moves, but might leave your own king in check.
Legal move	A pseudo-legal move that doesn't leave your own king in check.
FEN	A one-line text format that describes a whole chess position.
Perft	Counting every position reachable at a given depth, to test move generation.
Evaluation	A number saying how good a position is. Positive is good for White.
Centipawn	One hundredth of a pawn. The unit evaluations are measured in.
Minimax	Searching ahead, assuming each side always picks its best move.
Alpha-beta	Minimax that skips lines which can't change the result. Same answer, much faster.
Horizon effect	Misjudging a position because the search stopped in the middle of a trade.
Transposition	The same position reached through a different order of moves.
UCI	Universal Chess Interface: the text protocol engines use to talk to chess apps.